



UNIVERSIDAD
DE GRANADA

Este documento está protegido por la Ley de Propiedad Intelectual ([Real Decreto Ley 1/1996 de 12 de abril](#)).
Queda expresamente prohibido su uso o distribución sin autorización del autor.

Tecnologías Web

3º Grado en Ingeniería Informática



Proyecto Web

Gestión de reservas de aulas

1. Descripción general.....	2
2. Estructura del sitio web.....	2
3. Descripción del sistema en función del tipo de usuario.....	3
4. Información editable del sitio web.....	5
5. Salas.....	5
6. Reservas.....	6
7. Log del sistema.....	9
8. Operaciones de administración de la BBDD.....	9
9. Sobre el diseño, implementación y evaluación del proyecto.....	10
10. Publicidad y autenticidad de la práctica.....	15
11. Entrega de la práctica.....	16
12. Prácticas en equipo.....	16

© Prof. Javier Martínez Baena
Dpto. Ciencias de la Computación e I. A.
Universidad de Granada



Departamento de
Ciencias de la Computación
e Inteligencia Artificial

1. Descripción general

El proyecto plantea la creación de una aplicación web para la gestión de aulas en un centro docente. La web dispondrá de distinta funcionalidad dependiendo del tipo de usuario que la ve en cada momento.

- Quienen visitan la web sin estar identificados podrán ver información sobre las aulas disponibles y las reservas hechas. Cualquier usuario puede registrarse en el sitio web mediante un formulario. De esta forma podrá acceder al sitio identificándose.
- Los usuarios registrados podrán hacer reservas de salas y añadir comentarios en la web.
- Por último, la aplicación admitirá un tipo de usuario que tendrá la facultad de administrar las salas disponibles o tareas más propias de administración del sistema como, por ejemplo, hacer copias de seguridad de la base de datos.

Se recomienda leer este documento por completo antes de iniciar la práctica para tener una visión global de lo que se pide y, así, se pueda abordar el diseño con suficiente previsión. Este documento contiene una descripción del sistema como podría hacerla un potencial cliente e incorpora detalles técnicos y sugerencias de implementación. También se incluyen aclaraciones en relación con la elaboración, entrega y evaluación del proyecto.

Aunque puede planificar la elaboración del proyecto como estime oportuno, se indican a continuación una lista de tareas ordenadas que le pueden ayudar a comenzar a trabajar desde este mismo momento teniendo en cuenta la secuenciación de contenidos de la asignatura:

1. Diseño del sitio. Debería comenzar realizando bocetos/mockups para todas las páginas del sitio (o al menos las más relevantes). Esto le dará una visión general de lo que quiere hacer y posiblemente le ayude a detectar necesidades o detalles que va a necesitar desarrollar. En un proyecto real le permitirá también intercambiar información con el cliente ya que este, a partir de los bocetos, se hará una idea más clara de cómo podría quedar el producto que le ha encargado.
2. Maquetación. Realización del diseño gráfico del sitio (HTML+CSS). Desarrolle código HTML y CSS para la estructura general que van a tener todas las páginas así como de formularios, listados, menús, etc. Al finalizar esta etapa debería tener un fichero CSS con todo lo necesario para crear su sitio (incluso nuevas páginas) así como trozos de código HTML que más adelante integrará con PHP para la generación dinámica de todas las páginas.
3. Diseño e implementación con PHP. Diseñe la estructura de su proyecto con PHP y comience la implementación. Aún no se recupera información de la base de datos pero puede simularlo con datos ficticios para disponer de un prototipo inicial con algunas funcionalidades de la aplicación web. Integre el código HTML+CSS del paso anterior.
4. Una vez tenga una versión inicial del proyecto puede probar que funciona correctamente en `void.ugr.es`. Recuerde que si deja las pruebas en el servidor de prácticas para el último momento algo podría fallar y su calificación global podría ser "cero".
5. Diseño de la BBDD. Esto podría hacerse en paralelo con pasos anteriores. Pensar en la información que va a manipular su aplicación desde los inicios le puede ayudar a construir mejor cada página y evitar deshacer trabajo previo.
6. ... no olvide avanzar en la documentación conforme avanza en el proyecto.
7. Para hacer la integración de la base de datos con el prototipo que ya tiene puede comenzar con la parte de gestión de usuarios: formularios para crear nuevos usuarios, obtener listados, edición y borrado.
8. Una vez hecho esto compruebe que la aplicación sigue funcionando bien en `void.ugr.es`.
9. Desarrolle los módulos para autenticar usuarios (*login/logout*) e intégrelo en su código.
10. Continúe el desarrollo del resto de aplicación.
11. Añadir funcionalidad marcada con **[+OPC+]**: *JavaScript*, *AJAX*, etc.

2. Estructura del sitio web

Debe implementar la web con un diseño coherente para todas las páginas, es decir, todas mantienen una estructura común que constará, al menos, de:

- Encabezado. En esta parte puede incluir elementos tales como el nombre del centro

docente, alguna imagen o logotipo, etc. Parte de esta información procederá de la descripción del centro docente (ver sección 4).

- Menú de navegación.
- Pie de página.
- Zona principal para mostrar contenidos de cada página.
- Zona lateral secundaria.

Además, la web también debe contener un elemento que permita que un usuario se identifique (*login*) o bien, si ya está identificado, que salga del sistema (*logout*) y que consulte o modifique sus datos personales. Este elemento puede ubicarse en el encabezado, en la zona lateral o en alguna otra zona específica para ello. En cualquier caso formará parte de la estructura general de la web y debe aparecer en todas las páginas.

El menú de navegación variará dependiendo del tipo de usuario que está viendo la web. Habrá algunas páginas que puede ver cualquier usuario y otras que solo pueden ver algunos usuarios. Concretamente, las páginas disponibles para todos los usuarios son estas:

- Página inicial. Se trata de una página con una parte de contenido estático que sirve de entrada al sitio web con algún eslogan, imágenes o textos de bienvenida. También incluirá la descripción del centro docente (ver sección 4).
- Página con las salas disponibles. Es una página que muestra todas las salas de las que se dispone en el edificio incluyendo los datos de cada una de ellas (nombre, capacidad, fotos, comentarios, etc.).
- Página con el estado de las reservas. Debe incluir información sobre las reservas de todas las salas. Más adelante se propone algún ejemplo sobre el diseño que podría tener.

Zona lateral secundaria

Es muy frecuente que los sitios web dispongan de una zona lateral secundaria con información de segundo nivel. En nuestro caso, en esta zona secundaria debe mostrar la siguiente información:

- Número total de aulas del edificio.
- Capacidad total, es decir, número total de puestos disponibles en el edificio.
- Para el día de hoy: número de salas reservadas.

Esta zona secundaria es un buen lugar para contener el formulario de *login/logout* aunque puede ubicarlo en otra parte si lo estima oportuno.

Pie de página

En el pie de página del sitio mostrará la siguiente información:

- Autor/es de la aplicación.
- Enlace al documento pdf con la documentación de la práctica.
- Enlace al fichero de datos para la restauración de la BBDD.

3. Descripción del sistema en función del tipo de usuario

Como se ha indicado, los tipos de usuario admitidos en el sistema son los siguientes:

- Anónimo. Usuarios que navegan por internet y visitan el sitio web.
- Registrado. Usuarios que se han registrado en la aplicación.
- Administrador. Son los administradores del sistema web.

Dependiendo del tipo de usuario la información que muestra el sistema variará. También variará el menú de navegación mostrando solo aquellos enlaces que pueda visitar el usuario.

El sistema permitirá que haya un número indeterminado de usuarios de cada tipo, es decir, puede haber múltiples administradores y múltiples usuarios registrados.

Necesitará una tabla en la base de datos para almacenar la información de todos los usuarios del sistema y debería incluir un campo en el que indique el rol de cada usuario.

3.1. Usuario anónimo

Se trata de un visitante cualquiera que no ha hecho *login* en el sitio web. Para este tipo de usuario se incluirá una página para que pueda hacer el autoregistro en el sistema. La información mínima que se debe solicitar es la siguiente:

- Nombre
- Apellidos
- DNI
- Email (se utilizará para identificarse/*loguearse*).
- Clave
- Fotografía

Registro de usuarios

Para hacer el registro de usuarios anónimos deberá disponer un formulario de captura de datos. El sistema debe validar los datos de acuerdo a las siguientes indicaciones:

- El nombre no puede estar vacío.
- Los apellidos no pueden estar vacíos.
- Por simplicidad, solo se admitirán DNI españoles y deberá comprobar que la letra es correcta. El formato consistirá en una secuencia de 8 dígitos seguidos de una letra mayúscula (sin espacios ni guiones). El algoritmo utilizado es este:
<https://www.interior.gob.es/opencms/ca/servicios-al-ciudadano/tramites-y-gestiones/dni/calculo-del-digito-de-control-del-nif-nie/>.
- El *email* será una dirección adecuada. Use la función `filter_var()`.
- La clave debe contener al menos 4 caracteres alfanuméricos. No hay más restricciones.
- La foto no tiene ningún requisito.

3.2. Usuario registrado

Cuando un usuario anónimo ha hecho el autoregistro su información se almacenará en la base de datos (con rol de cliente) y, desde ese momento, podrá identificarse en el sistema usando su *email* y su clave. Una vez identificado ya no aparecerá la página de autoregistro.

La información (páginas) que estos usuarios pueden ver es la siguiente:

- Pueden acceder a sus datos personales. Pueden editar solo algunos de ellos: *email*, clave y fotografía. El nombre, apellidos y DNI no pueden ser modificados.
- Pueden realizar las siguientes operaciones CRUD de reserva de salas:
 - Realizar una nueva reserva.
 - Ver las reservas realizadas (solo las suyas).
 - Cancelar las reservas (solo las suyas).
 - No pueden editar una reserva. Si necesitan hacer una modificación deberán borrar la reserva antigua y crear una nueva.

Además, los usuarios registrados también pueden hacer comentarios (texto) a cada una de las salas (tanto si las ha reservado como si no). Pueden hacer un número ilimitado de comentarios. Los comentarios, una vez hechos, no pueden editarse ni eliminarse.

3.3. Usuario administrador

Este tipo de usuario podrá realizar también las tareas específicas de los registrados (crear nuevas reservas o añadir comentarios).

Además, realizará las siguientes tareas de administración:

- Operaciones CRUD sobre las salas del edificio: Ver las salas disponibles, crear nuevas salas, editarlas y borrarlas.
- Operaciones CRUD sobre los usuarios. Ver usuarios del sistema, añadir nuevos usuarios, editarlos y borrarlos. Puede cambiar el rol de los usuarios y puede modificar todos sus datos.

- Operaciones CRUD sobre las reservas de cualquier usuario. Los usuarios registrados pueden realizar operaciones CRUD solo sobre sus propias reservas mientras que los administradores pueden hacerlo sobre las reservas de cualquier usuario.
- Operaciones sobre la base de datos:
 - Hacer un *backup* de la BBDD.
 - Restaurar la información de la BBDD desde un *backup* previo.
 - Reiniciar la BBDD. Borrar toda la información (pero no las tablas).
- Visualizar el *log* del sistema.

Cuando se reinicia la base de datos, debe tener cuidado con crear o mantener un usuario administrador al menos para poder acceder al sistema y poder crear nuevos usuarios.

4. Información editable del sitio web

Los usuarios administradores podrán editar los siguientes elementos desde la aplicación:

- Nombre del centro docente.
- Logotipo del centro docente.
- Descripción del centro.
- Horario de apertura diaria. Hora de comienzo y hora de final.

Esta información se mostrará en los encabezados de todas las páginas (nombre y logotipo) y en la página de inicio del sitio web (descripción). El horario de apertura se utilizará para limitar las horas en las que se pueden hacer reservas.

5. Salas

Cada sala debe contener la siguiente información:

- Nombre de la sala. Cadena de texto arbitraria.
- Ubicación. Cadena de texto.
- Número de puestos disponibles. Número entero.
- Descripción. Cadena de texto.
- Fotografías. Pueden ser cero o más fotografías (sin límite).
- Reservable.

Todos los usuarios del sistema pueden ver listados con la información de todas las salas. Los administradores son los únicos que pueden editar esta información (añadir, editar o borrar).

Además, los usuarios registrados y los administradores pueden añadir comentarios (texto) a cada una de las salas. El número de comentarios de cada sala es ilimitado. Estos también se mostrarán a ver la información de cada sala.

El campo reservable se utilizará para saber si la sala se puede reservar o no por los usuarios registrados. En caso de que no sea reservable estos podrán verlas pero no podrán reservarlas. Sin embargo, los administradores sí podrán crear reservas también en estas salas.

5.1. Las fotografías de las salas

Una sala puede contener cero o más fotografías pero no se sabe a priori cuántas puede haber. El inconveniente es que, al desconocer este dato, no es posible crear un formulario con un determinado número de controles de tipo `input file`. Puede seguir varias estrategias para resolver este problema:

- Añadir fotografías de una en una:
 - Existirá un formulario para dar de alta una nueva sala. En este solo se consignan los aspectos textuales (nombre de la sala, descripción, etc).
 - Será cuando se entre a editar el contenido de la sala cuando se muestre una página con varios formularios independientes: uno para los datos textuales y otro para las fotografías. El formulario de las fotografías mostrará el listado de fotografías ya almacenadas (cada una con la posibilidad de ser eliminada) y un botón para añadir una nueva.

- Usar el atributo **multiple** en el control **input file** para subir varios ficheros a la vez.
- Usar **AJAX** para subir ficheros.

Para esta práctica puede usar la opción que desee siempre y cuando permita añadir un número indeterminado de fotografías a una sala así como eliminarlas de forma individual.

Al subir fotografías debería comprobar que lo que sube el usuario es realmente una fotografía y que tiene algún formato adecuado (o hacer la conversión si fuese necesario). En cuanto a cómo almacenar las fotografías en el servidor puede considerar varias opciones:

- Almacenar las fotografías en ficheros y registrar en la base de datos únicamente el nombre de dichos ficheros. En este caso debe tener cuidado con los nombres de los ficheros para evitar caracteres extraños, accesos a otras carpetas del sistema o nombres duplicados. Los ficheros se almacenarían en alguna carpeta accesible desde internet que, en principio, no tendría porqué estar protegida frente a accesos indebidos.
- Almacenar las fotografías en la base de datos (campo de tipo **BLOB**). En este caso se garantiza la privacidad de las mismas (por si fuese importante). Evitamos así problemas con nombres de ficheros, duplicidades, etc. Cuando se recupera esta información de la BBDD tenemos varias alternativas para mostrarla en la web.
 - La primera de ellas es incrustar el contenido de la imagen dentro del **tag img** de HTML evitando así el acceso a ficheros adicionales. Por contra, el documento HTML es mucho más pesado, se impide la recuperación en paralelo de varias fotografías con HTTP y se impide también el uso de la caché del navegador para fotografías que ya se cargaron con anterioridad.
 - La segunda forma sería que, una vez recuperada la imagen de la BBDD, el servidor la almacenaría en un fichero temporal en disco y en el documento HTML se colocaría la URL de dicho archivo. Una vez descargado el archivo de imagen este no debería seguir siendo accesible desde internet. Esta opción complica un poco la lógica de programación pero aceleraría la carga de la página.
 - Otra forma sería implementar un script PHP en el servidor al que se le solicita una imagen y este devuelve el fichero con dicha imagen. De esta forma evitaríamos crear ficheros de acceso temporal en el servidor.
- Una tercera alternativa sería almacenar las fotografías en ficheros aunque en carpetas no accesibles desde internet. Cuando se accede a una de esas fotografías, el sistema puede copiar el fichero en una carpeta accesible desde internet y enviar su URL. De esta forma mantenemos las ventajas de ambos métodos, aunque se complica también la lógica de la aplicación.

En este proyecto se valorarán todas las opciones por igual y no se tendrán en cuenta los factores de eficiencia o privacidad comentados. Por simplicidad se recomienda almacenar las imágenes en la base de datos (campos de tipo **BLOB**).

6. Reservas

Cuando un usuario registrado hace una reserva lo que debemos almacenar es esto:

- Usuario que hace la reserva.
- Sala reservada.
- Motivo de la reserva.
- Fecha de la reserva.
- Horas de inicio y final de la reserva.

Para las horas se pueden establecer slots de tiempo de 1/2 en 1/2 hora o de hora en hora, por ejemplo. Siempre estarán dentro del horario de apertura (ver sección 4).

La reserva se realizará siguiendo estos pasos:

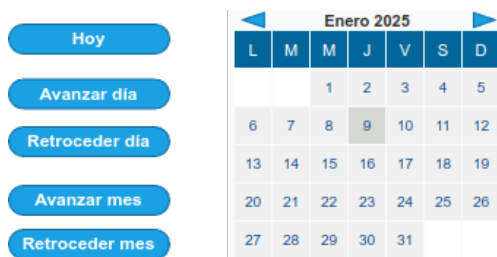
- El usuario seleccionará la fecha en la que quiere hacer la reserva.
- El sistema mostrará todas las salas junto con información sobre las reservas de esa fecha.
- El usuario elegirá la sala y las horas de inicio y fin de la reserva.
- Cuando envíe la información, el sistema comprobará que la sala está libre. Si es así la reservará y si no mostrará un mensaje al usuario y le pedirá que haga una nueva reserva.

6.1. Crear reservas

Para la creación de reservas el sistema mostrará una página con dos elementos principales. Por una parte tendrá un bloque para poder elegir la fecha que queremos consultar. Este debe incluir:

- Calendario del mes.
- Botones o enlaces para: ir a "hoy", avanzar un día, retroceder un día, avanzar un mes y retroceder un mes.

La siguiente figura muestra un ejemplo:



El segundo bloque mostrará la información de las reservas de la fecha elegida. En la siguiente figura puede ver un ejemplo:

Jueves 9/1/2025	8:00	8:30	9:00	9:30	10:00	10:30	11:00	11:30	12:00	12:30	13:00	13:30	14:00	14:30	15:00	15:30
Aula 0.1																
Aula 0.2			TW (Javier)													
Aula 0.3									MP							
Polivalente																
Lab 2.1									FR (Pedro)							
Lab 2.2																
Lab electrónica																
Sala de reuniones																
Salón de actos									Conferencia sobre IA							
Módulo A - A1																
Módulo A - A2																

En el listado debe aparecer la siguiente información:

- Fecha (se corresponderá con la marcada en el calendario anterior).
- Una columna por cada *slot* de tiempo disponible.
- Una fila por cada sala disponible.
- Se marcarán las celdas que se correspondan con alguna reserva y se distinguirán las reservas propias de las de otros usuarios. En la figura de ejemplo se muestran en verde las propias y en azul el resto.
- Las salas que no sean reservables también se marcarán de alguna forma. En el ejemplo se muestran con fondo gris.
- En las reservas se mostrará el motivo y el usuario que las hace.

Desde esa tabla el usuario podrá ver la información de cada sala. Puede usar algún tipo de *pop-up* al pasar el ratón por encima del nombre, usar *JavaScript* o *AJAX* o bien puede abrir una nueva página con la información.

Desde esta misma página el usuario también podrá hacer una nueva reserva. Puede pulsar sobre las celdas para marcar las horas de la reserva o puede pulsar algún botón o incluso el nombre de la sala para abrir un formulario que pregunte los datos de la reserva.

Una vez que se hayan indicado los datos de la reserva, el sistema debe verificar que es factible.

6.2. Listados y búsqueda de reservas

Debe incluir una página que facilite la búsqueda de reservas que cumplan con determinados criterios. Esta página tendrá dos bloques. El primero será un formulario para indicar los criterios de búsqueda y ordenación similar al siguiente:

Formulario de búsqueda de reservas:

- Motivo:** Campo de texto con el valor "texto".
- Fecha inicial:** Campo de fecha con el valor "fecha1".
- Fecha final:** Campo de fecha con el valor "fecha2".
- Usuario:** Selector desplegable con "-cualquiera-" seleccionado. La lista desplegada muestra: Javier, Pedro, Antonio (seleccionado).
- Sala:** Selector desplegable con "-cualquiera-" seleccionado. La lista desplegada muestra: Aula 0.1, Aula 0.2, Aula 0.3 (seleccionado), Polivalente, Lab 2.1.
- Ordenar por:** Botones de radio. "Fecha y hora" está desactivado, "Sala" está activado.
- Ítems por página:** Campo de texto con el valor "5".
- Botón:** "Aplicar criterios de búsqueda y ordenación".

El usuario puede dar valor a cero o más campos. Si en uno de los campos no indica ningún valor quiere decir que no es incorporado a la búsqueda y que, por tanto, es válido cualquier valor para el mismo. Por ejemplo, si deja vacío el campo "motivo" quiere decir que no filtra los resultados con ese campo. Concretamente, los campos que debe incluir son los siguientes:

- Motivo.** Selecciona solo aquellas reservas cuyo motivo contenga la cadena escrita en este campo. Si se deja vacío no lo incluye en la búsqueda.
- Fecha inicial y fecha final.** Selecciona las reservas que se han hecho en el rango dado. Si no se indica fecha inicial se buscan solo las que son anteriores a la fecha final. Si no se indica fecha final busca solo las posteriores a la fecha inicial. Si no se indica ninguna de las dos fechas no incorpora este criterio en la consulta.
- Usuario.** Debe mostrar una lista con todos los usuarios registrados en el sistema y mostrar únicamente las reservas de los usuarios indicados. Si no se indica ningún usuario no se incorpora este criterio y, por tanto, muestra las reservas de todos los usuarios. Tiene varias opciones para este control: puede ser un desplegable, puede ser un checkbox, puede ser un grupo de radio button, etc. Cabe la posibilidad de que la selección sea de un único elemento (desplegable o radio button) o de múltiples elementos (desplegable con la opción multiple o checkbox).
- Sala.** Es similar al control usuario, pero para seleccionar las salas sobre las que queremos hacer la consulta.

Observe que cuando se seleccionan múltiples controles se hace la búsqueda con la Y lógica de todos ellos. Es decir, la búsqueda debe cumplir con todos los criterios indicados.

Además de los criterios de búsqueda, debe añadir un control para decidir sobre el orden de las tuplas mostradas (ordenar por). Se pueden ordenar por fecha y hora (primero las más antiguas) o por sala (orden alfabético). Puede decidir uno cualquiera como valor por defecto.

El segundo bloque de esta página es la lista de tuplas que cumplen con los criterios de búsqueda y ordenación. La siguiente figura muestra un ejemplo:

Fecha	Horas	Sala	Usuario	Motivo
8/1/2025	9:30 - 11:30	Aula 0.2	Javier	TW
8/1/2025	10:30 - 11:30	Lab electrónica	Pedro	Examen de TOC
10/1/2025	13:30 - 14:30	Aula 0.3	Pedro	Conferencia



Este listado debe ser paginado, es decir, debe incluir algún tipo de barra al estilo de la que se muestra en la parte inferior para mostrar elementos de X en X. El número de elementos que se muestra en cada página es configurable. En estos ejemplos se ha incluido un campo "Ítems por página" en el formulario del primer bloque.

Uso de cookies

La primera vez que se carga esta página el formulario con los criterios de búsqueda tendrá todos

los campos sin rellenar por lo que el listado que se ve a continuación es el listado completo de reservas.

Al rellenar los campos de búsqueda y pulsar el botón de “Aplicar criterios ...” se actualizará el listado que aparece a continuación. Si más adelante regresamos a esta página, en lugar de que aparezca el formulario de criterios de nuevo en blanco, queremos que conserve los últimos criterios de búsqueda aplicados. Para ello, debe utilizar *cookies* para almacenar todos los criterios de búsqueda.

6.3. Operaciones CRUD sobre las reservas

Los usuarios registrados pueden listar y borrar sus reservas y los administradores pueden listar borrar las reservas de cualquier usuario. Puede aprovechar este mismo listado para incluir la funcionalidad CRUD necesaria.

Lo que se muestra en el ejemplo de la sección 6.2 sería para el caso de ser administrador. Para los usuarios registrados que no son administradores, este mismo código puede valer si omitimos el control “Usuario” (no se muestra o, si se muestra, se muestra con un único valor elegible -y elegido obligatoriamente- que sería el del usuario registrado).

Para la operación de borrar reservas bastaría con añadir un botón o enlace al lado de cada tupla de manera que si se pulsa se elimina dicha reserva.

Las reservas no son editables o modificables por ningún tipo de usuario.

7. Log del sistema

Se debe mantener un registro (*log*) con los eventos principales que se van produciendo en la aplicación:

- Cada vez que se identifica un usuario.
- Cada vez que se registra un nuevo usuario en el sistema.
- Cada vez que un usuario hace alguna acción que modifique la BBDD (inserciones, editados o borrados de datos).
- Cada vez que cierra la sesión un usuario identificado.
- Intentos de identificación erróneos.

Este registro se mantendrá en una tabla de la base de datos y la información que debe almacenarse es:

- Hora y fecha del evento.
- Breve descripción. Una cadena que describe lo que ha ocurrido.

Cualquier usuario administrador podrá ver un listado con las tuplas de esta tabla ordenadas por fecha (de más recientes a más antiguas).

8. Operaciones de administración de la BBDD

Los usuarios administradores podrán realizar estas operaciones para hacer un mantenimiento básico de la base de datos:

- Obtener una copia de seguridad de la BBDD. La aplicación web enviará al usuario un fichero de texto plano con las cláusulas SQL que permiten la restauración completa de la base de datos incluyendo instrucciones de tipo **DROP**, **DELETE**, **CREATE**, **INSERT**, etc. Sería un fichero al estilo de lo que podríamos obtener con un comando `mysqldump` aunque debemos generarlo desde nuestro código PHP ya que no tenemos privilegios de ejecución de comandos en la *shell* del sistema.
- Restaurar la BBDD a partir de una copia de seguridad previa. El administrador envía un fichero de texto plano como el obtenido en el punto anterior y la aplicación ejecutará las cláusulas contenidas en él.
- Reiniciar la BBDD. La aplicación borrará la información almacenada en el sistema y la dejará con todas la tablas necesarias para la aplicación creadas pero sin ninguna tupla almacenada.

Tanto en el caso de restauración como en el de reinicio deberá tener en cuenta que para que el

sistema esté operativo debe existir algún usuario de tipo administrador para comenzar a crear usuarios recepcionistas y que estos, a su vez, puedan crear habitaciones, etc. Por tanto, debe asegurarse de que exista un administrador en ambos casos.

IMPORTANTE: si no ha podido implementar la restauración (o el reinicio) con éxito NO los incluya en la aplicación ya que si el profesor accede a ellos durante la corrección podría dejar inservible el sistema. Si eso ocurre la calificación global de la práctica podría ser **cero. Por tanto, asegúrese de que estas operaciones funcionan correctamente y haga pruebas suficientes en void.ugr.es antes de incluirlas. Si no está seguro de que funcionen bien o no funcionan: no las incluya e indíque en la documentación el motivo por el que no se han implementado.**

Observe que la operación de reinicio puede ser de utilidad también para hacer la instalación de la aplicación en el servidor web. Para ello, tras copiar los ficheros PHP, HTML, CSS, etc. al servidor por primera vez deberá crear las tablas de la base de datos antes de comenzar a ejecutar la aplicación y crear un primer usuario administrador. Esto puede hacerlo de forma manual mediante alguna aplicación como `phpmyadmin` o `adminer`. Sin embargo, sería mucho más cómodo si lo hiciese su propia aplicación de manera que tras copiar los ficheros indicados, la primera vez que se accede a `index.php` (o a cualquier *script* de su aplicación) y comprobar que aún no existen las tablas, su código PHP las cree. Este código es en esencia el mismo que ejecutaría en el proceso indicado de reinicio de la BBDD. Esto simplificaría el proceso de implantación de su código en cualquier servidor web.

9. Sobre el diseño, implementación y evaluación del proyecto

Aunque esta asignatura presupone que ya tiene conocimientos suficientes de programación, tenga en cuenta que de cara a la evaluación se tendrán en cuenta aspectos básicos tales como: el estilo de programación, la calidad técnica del código, la documentación interna o comentarios en el código, etc. Se enumeran a continuación algunas cuestiones genéricas a tener en cuenta:

- En la asignatura no se instruye sobre el uso de ningún tipo de *framework*. Previa consulta al profesor, se podrá permitir el uso de algún *framework* de CSS siempre y cuando quede acreditado que el alumno tiene los conocimientos impartidos en la asignatura sobre CSS además de los conocimientos adecuados sobre el *framework* en cuestión. En ningún caso se permite usar *frameworks* de PHP ni de *JavaScript*.
- Se deberán utilizar elementos de HTML y CSS estándares y con suficiente implantación en los navegadores actuales. La práctica se evaluará en versiones actualizadas de Firefox y/o Chrome.
- La aplicación debe funcionar correctamente en el servidor `void.ugr.es`. En caso de que falle la práctica, esta podrá tener una calificación global de "**cer**o" independientemente de otros aspectos de la misma.
- La base de datos debe contener suficientes datos de prueba como para poder comprobar el correcto funcionamiento de todas las páginas del sitio. Así mismo, en la documentación debe incluirse el correo electrónico y clave de todos los usuarios registrados en el sistema así como el rol de cada uno de ellos para poder probar la aplicación. El no cumplimiento de este punto puede implicar una calificación global de "**cer**o".
- Se deberá prestar especial atención al cumplimiento de todas las normas explicadas en este documento. El no cumplimiento de las mismas podrá implicar una calificación global de "**cer**o".

A continuación se desarrollan otros detalles importantes que se tendrán en cuenta para la evaluación de este trabajo. Si tiene dudas sobre algún punto no dude en consultar al profesor.

9.1. Organización del código

La organización interna del sitio es libre pero se tendrán en cuenta los siguientes aspectos:

- Puede usar un paradigma procedural, orientado a objetos o mixto. Independientemente del paradigma, la modularización del código en clases/métodos/funciones, ficheros y directorios ha de ser razonable y coherente con lo estudiado en la asignatura.
- Debe procurar tener una arquitectura que procure una separación razonable entre distintas facetas de la aplicación: vistas, modelos y controladores.

- Para ver las distintas páginas de la aplicación puede optar, indistintamente, por:
 - Tener un único fichero (**index.php**) para gestionar el sitio completo. Ese *script* sirve para generar todas las páginas de la aplicación. Para determinar qué página se está solicitando puede:
 - Usar parámetros **GET** para ver los distintos contenidos.
 - Usar algún tipo de enrutador que analice la URL y determine el contenido a ver.
 - Tener múltiples *scripts* PHP para ver las distintas páginas del sitio (no se recomienda).

En relación con la reutilización de código se tendrá muy en cuenta lo siguiente:

- Debe evitar la técnica del "copia/pega", que será evaluada de forma negativa (e incluso podría implicar obtener una nota que le impida superar el proyecto). En caso de que utilice múltiples *scripts* para ver las distintas páginas del sitio (en lugar de un único **index.php**) se admitirá un copia/pega con el esqueleto básico de una página PHP siempre y cuando los elementos que incluya sean mínimos y correctamente modularizados. Por ejemplo, para incluir código **HTML** o **PHP** o para hacer algunas llamadas a funciones **PHP** que maqueten la página o comprueben la autenticación de usuarios. Si tiene dudas consulte con el profesor.
- Será frecuente que tenga que mostrar, para ciertos procesos, un formulario con distintas variantes: vacío, con valores previos o con controles deshabilitados. Para estos casos debe reutilizar código, es decir, disponer de una única función o método que genere el formulario en lugar de distintas funciones para crear distintas versiones del mismo formulario.

9.2. La base de datos

Aunque no se pide mucho detalle en el diseño de la base de datos, sí se requiere que en la documentación del proyecto se incluya, al menos, y de forma breve:

- Un modelo E-R.
- El modelo relacional que se obtiene a partir del modelo E-R.
- Breve descripción de las tablas obtenidas.

El modelo relacional explicado en la documentación debe corresponderse con el implementado en la aplicación. El no cumplimiento de este punto puede implicar una calificación global de "**cero**".

9.3. Datos de prueba

Para poder hacer la evaluación del proyecto, la base de datos deberá contener información útil cuando se entregue.

- No incluir esta información puede llevar a tener una calificación global de "**cero**" en la práctica.
- Dicha información debe tener sentido y debe servir para probar toda la funcionalidad implementada. Lo más apropiado sería inspirarse en un ejemplo real aunque también pueden utilizarse datos ficticios, en cuyo caso no se permitirá usar textos sin sentido o muy genéricos (es decir, no se deben usar nombres o textos como "aheitc4mw2xsko" o "Texto de prueba").
- En la entrega de la práctica debe incluir un fichero de restauración de la BBDD que contenga todas las cláusulas que permiten borrar (**DROP**) y crear (**CREATE**) las tablas de la aplicación así como las inserciones (**INSERT**) de los datos de prueba. Debe incluir un enlace a este fichero en el pie de página de la aplicación.

Para asegurar una corrección homogénea, se proporcionan a continuación listados con lo que debería incluir y que el profesor espera encontrar (como mínimo). Se indican solo los campos más relevantes pero el resto también deben contener datos.

Usuarios registrados

Email	Clave	Rol
admin@void.ugr.es	admin	Administrador
javier@void.ugr.es	javier	Administrador
pedro@void.ugr.es	pedro	Usuario

anita@void.ugr.es	anita	Usuario
maria@void.ugr.es	maria	Cliente

Aulas

Nombre	Capacidad	Reservable	Nº fotografías
Aula 0.1	60	Sí	1
Aula 0.2	70	Sí	0
Aula 0.3	80	Sí	0
Lab electrónica	18	Sí	1
Polivalente	30	Sí	2
Salón de actos	150	No	3
Salón de grados	80	No	>= 3
Lab 2.1	38	Sí	>= 3

- El resto de campos también deben tener contenido (descripción, ubicación).
- Las fotografías deben tener distintos tamaños y relación de aspecto.
- Se pueden reutilizar fotos para varias salas.

Reservas

- Debe tener al menos 15 reservas ya hechas en fechas que han de estar en el rango 1/1/2025 - 30/7/2025.
- Al menos 4 de las reservas serán de las salas marcadas como no reservables.
- Deberá tener en algún día más de 5 reservas.
- Todos los usuarios han de tener alguna reserva hecha excepto el usuario maria, que no tendrá ninguna.
- Debe incluir en la documentación un listado (legible) con todas las reservas incluidas para poder hacer las comprobaciones durante la corrección.

9.4. Diseño del sitio

El diseño de sitios web es un tema complejo que no se aborda en la asignatura. En esta práctica, tanto el diseño gráfico como el maquetado del sitio son libres. La evaluación tendrá en cuenta, sobre todo, la parte técnica y aspectos funcionales del sitio (uso de las tecnologías estudiadas, usabilidad, etc).

- La interfaz de la aplicación debe ser homogénea en todas las páginas. Se valorará muy negativamente el cambio de diseño (sin justificación) en diferentes páginas del sitio.
- La interfaz debe ser funcional: tamaño proporcionado del texto y de otros elementos, colores "razonables", elementos bien posicionados, que no haya solapamientos, etc. La parte estética o artística pasa a un segundo plano aunque puede influir en la calificación de la siguiente forma:
 - Caso 1: si es muy deficiente, impide un uso razonable de la web o demuestra un muy bajo conocimiento de las tecnologías empleadas, será valorada de forma negativa pudiendo llevar a una calificación global de suspenso.
 - Caso 2: si es un diseño muy bueno con un aspecto profesional se valorará positivamente.
- Será preferible que utilice un diseño fluido o mixto fijo/fluido. En cualquier caso, la página deberá visualizarse correctamente en un monitor con resolución FullHD y permitiendo que el ancho del navegador pueda disminuirse aproximadamente hasta la mitad de la pantalla.
- El uso de un diseño adaptable se valorará positivamente. [+OPC+]

La práctica se corregirá con versiones actualizadas de Firefox y/o Chrome. Se recomienda que el

alumno la pruebe en ellos para asegurarse de que todo es correcto.

9.5. Formularios del sitio

Todos los formularios del sitio deben cumplir las siguientes normas:

- Deben ser de tipo *sticky* cuando proceda. Por ejemplo en todas las operación de edición.
- La validación de datos del formulario debe hacerse en el servidor (**PHP**) en cualquier caso.
- Será **obligatorio** que todos los formularios incluyan el atributo **novalidate**. Caso de no hacerse se penalizará por no poder comprobar la validación en el servidor.
- Cada dato deberá ser validado de acuerdo a su tipo. Por ejemplo, la validación de un *email* será distinta que la de un apellido. Cuando se detecten errores en un control de entrada deberá informar al usuario de los mismos. Esta información debe mostrarse de manera que al usuario le sea fácil identificarlos y corregirlos (por ejemplo junto al control erróneo o con un orden lógico). Además, si hubiese más de un error se mostrarán todos los mensajes de error en el formulario (y no solo uno o algunos de ellos). También puede ayudarse del atributo **placeholder** para mejorar la entrada de datos.
- La validación en el cliente (*JavaScript*) es secundaria y se hace solo para mejorar la usabilidad del sitio.
- En la medida de lo posible, los datos de un proceso deben recogerse en un único formulario. No se admitirán soluciones que obliguen a un usuario a pasar por varios formularios de forma innecesaria (para realizar un proceso que podría haberse hecho con un único formulario). Por ejemplo, para registrar un cliente se deben pedir todos los datos en un único formulario en lugar de pedir el nombre, en un segundo formulario los apellidos, en un tercer formulario el DNI, etc. (se exceptúan las fotos por los motivos ya explicados).
- No deberá mostrar o solicitar información que sea interna del sistema (por ejemplo códigos, claves primarias de la BBDD, *hash* de *passwords*, etc).
- La modificación o inserción de datos, como norma general, serán procesos que pidan confirmación, es decir, que necesitarán de:
 - Una primera página que muestra el formulario para editar o añadir información nueva.
 - Una segunda página que pedirá confirmación de inserción o edición. En este paso la información no debe poder modificarse, simplemente se muestra para que el usuario sepa lo que va a modificar o insertar.
 - Una tercera página que informe del éxito o no de la operación.
- El borrado de datos, como norma general, también pedirá confirmación por lo que se necesita:
 - Un primer formulario que presenta la información a borrar y pide confirmación. En este caso, el formulario podría tener como único **input** el botón de **submit**, el resto de datos podrían ser **inputs** no editables, de tipo **hidden** o incluso tags HTML que no sean controles del formulario.
 - Una segunda página que informa del resultado de la operación.

En casos como estos en los que los procesos de formulario necesitan varios pasos para completarse debe tener en cuenta que la información que se rellena en el primer formulario se envía normalmente con el método **POST**. Esa información llega al segundo formulario (o página) para mostrarla pero sin posibilidad de edición. En este caso es probable que use controles de tipo **input** con el atributo **disabled** o **readonly** activado (se pueden ver pero no modificar). Debe tener en cuenta que al enviar ese segundo formulario es posible que estos controles no sean enviados por lo que deberá estudiar la forma de que lleguen, si fuese necesario, al tercer formulario que es el que realiza la operación de inserción o modificación en la BBDD.

Cuando los atributos se marcan como **disabled** para que el usuario pueda verlos y no modificarlos estos no se envían al hacer **submit**. Si se marcan con **readonly** sí se envían pero hay algunos que no admiten este atributo (**checkbox**, **radio**, **select**, ...). Es decir: hay situaciones en las que no es trivial "retransmitir" valores entre formularios. En estos casos tenemos distintas alternativas para resolver el problema:

- Además de los controles con el atributo **disabled** podemos añadir otros controles de tipo **hidden** con la información que deseamos enviar y que no se enviará con el control **disabled**. Por ejemplo, en un formulario de edición de datos de un usuario podemos mostrar un **input**

de tipo `text+disabled` para que el usuario vea su nombre (sin posibilidad de modificarlo) y un segundo `input` de tipo `hidden` con el valor (`value`) que queremos enviar para el nombre (el atributo `value` coincidirá en ambos campos pero el `id` debe ser distinto). Esta solución obliga a que todas las etapas intermedias del proceso sean formularios y que todos ellos incluyan controles de tipo `hidden` para ir reenviando información de unos a otros.

- Una segunda solución pasa por utilizar variables de sesión. Cuando se envía la información en el primer formulario esta se almacena en variables de sesión y éstas estarán disponibles para todos los pasos siguientes sin necesidad de enviar la información múltiples veces entre cliente y servidor. Esta solución admite que no todas las páginas del proceso sean formularios ya que la información siempre está en el servidor: es más versátil y eficiente.
- Como alternativa a las variables de sesión podemos hacer uso de tablas temporales en la base de datos o almacenamiento en ficheros pero esto es más complejo de implementar y no aporta ventajas en nuestro caso.

Adicionalmente debe tener en cuenta que los `input` de tipo `file` ni siquiera permiten que se rellene su atributo `value` para "reenviar" el fichero. En este caso la única solución es el uso de variables de sesión (o el uso de ficheros o bases de datos en el servidor).

9.6. Seguridad de la aplicación

En relación con la seguridad de la aplicación se harán comprobaciones detalladas sobre los siguientes aspectos:

- Saneado correcto de datos provenientes de formularios o de la base de datos para evitar inyección HTML.
- Consultas seguras a la base de datos para evitar inyección SQL.
- Almacenamiento de claves cifradas en la base de datos.
- Validación de formularios en el servidor. Los formularios deben tener activo el atributo `novalidate` para poder hacer estas comprobaciones.
- Accesos no permitidos a determinadas páginas de la aplicación. Se comprobará si un usuario de un determinado tipo puede acceder a páginas diseñadas para usuarios de otros tipos (por ejemplo: si un usuario anónimo puede acceder a la página de reservas o si un recepcionista puede acceder a la páginas de *backup* de la base de datos). Tenga en cuenta que para evitar este problema no basta con cambiar los menus de navegación ya que cualquier usuario puede escribir una URL en el navegador e intentar acceder a una página sin permiso.

9.7. Uso de JavaScript [+OPC+]

Puede utilizar *JavaScript* para realizar algunas mejoras del sistema que, aunque no afectan a la funcionalidad del mismo, sí mejorarían la experiencia de usuario. Algunas sugerencias:

- Se pueden ocultar o mostrar formularios o determinados ítems para facilitar la legibilidad de la página. Por ejemplo:
 - Los listados de salas pueden resultar un poco extensos si incluimos para cada una toda su información (nombre, capacidad, descripción y, sobre todo, fotografías). Se pueden mantener ocultas la descripción y las fotografías y habilitar un botón para mostrarlos y ocultarlos de manera que se mantenga en pantalla solo la información que el usuario considere útil.

Observe que la funcionalidad de la aplicación no varía en caso de que, por algún motivo, el navegador no pueda ejecutar el código *JavaScript*, simplemente vería toda la información.

- Se puede usar *JavaScript* para hacer la validación de datos en el cliente. Esto no nos exime de hacer la validación en el servidor, que es obligatoria en cualquier caso.

Si el navegador no puede ejecutar el código *JavaScript*, simplemente afectaría a la usabilidad pero no sería un problema para la aplicación.

- ...

Se recomienda dejar esta tecnología para el final del proyecto. Una vez finalizado se pueden añadir estas pequeñas mejoras con poco esfuerzo y, de paso, evitarán que hagamos nuestra aplicación dependiente de *JavaScript*.

Si procede, en la documentación deberá incluir un apartado explicando en qué puntos utiliza *JavaScript*.

9.8. Uso de AJAX [+OPC+]

El uso de tecnología AJAX no se considera obligatorio en este proyecto aunque su uso en alguna parte se valorará de forma positiva. Algunos lugares en los que sería apropiado su uso:

- Paginación de listados de resultados. Al pulsar sobre los botones del paginador la página se actualizaría de forma dinámica.
- Ampliación de información. Por ejemplo, al hacer un listado de salas podría mostrarse únicamente el nombre de las mismas y un botón para ampliar información. Al pulsar el botón se contactaría con el servidor para obtener los datos adicionales de la sala y se mostrarían en la misma página.
- Se puede mejorar la usabilidad en el formulario de edición/creación de salas permitiendo que, cada vez que el usuario selecciona un fichero con una fotografía, esta se muestre de forma dinámica (sin necesidad de pulsar el botón de enviar) y generando nuevos controles en el formulario que permitan borrarla o añadir nuevas fotografías.

Si procede, en la documentación deberá incluir un apartado explicando en qué puntos utiliza AJAX.

9.9. Documentación de la aplicación

Debe incluir en la entrega un único fichero en formato pdf que incluya, al menos, los siguientes elementos:

- Identificación de el/los alumno/s que ha/n realizado la práctica.
- Datos incluidos para probar la práctica:
 - Nombre de usuario y clave de los usuarios registrados en la aplicación web para poder probarla (indicando para cada uno el usuario, la clave y el rol).
 - Salas definidas.
 - Reservas realizadas.
- Nombre del fichero de restauración de la BBDD con datos de prueba.
- Diseño previo de la aplicación (mockups, wireframe, etc)
- Documentación sobre la BBDD (ver subsección anterior).
- Explicaciones técnicas que considere relevantes para la evaluación de la práctica o que quiera poner en valor por algún motivo.
- Listado de los ítems opcionales que haya incluido en su proyecto, en particular sobre *JavaScript* y AJAX.

Además, el código desarrollado (HTML, CSS, PHP, *JavaScript*) debe estar convenientemente comentado.

En el pie de página del sitio web incluirá un enlace a este fichero de documentación.

9.10. Rúbrica de evaluación

Para la corrección y evaluación del proyecto el profesor usará una rúbrica con una serie de ítems que reflejan todo lo indicado en este documento y algunos elementos adicionales como, por ejemplo, corrección de errores cometidos en prácticas anteriores, calidad del código, cumplimiento de las normas, corrección de la documentación, etc. Para cada ítem no conseguido se aplicará una penalización.

A lo largo del documento se ha marcado con [+OPC+] alguna funcionalidad. Estos elementos se considerarán de forma positiva por encima de 10. Es decir, se puede conseguir una calificación de 10 sin realizar ninguno de ellos pero si se realizan se valorarán positivamente y pueden compensar fallos en algunos otros ítems.

10. Publicidad y autenticidad de la práctica

Se recomienda no publicar la práctica (ni esta ni otras) en repositorios públicos para velar por su privacidad **hasta la finalización del curso académico**. Según la normativa de evaluación y de

calificación de los estudiantes de la Universidad de Granada:

Artículo 13. Desarrollo de las pruebas de evaluación

7. Los estudiantes están obligados a actuar en las pruebas de evaluación de acuerdo con los principios de mérito individual y autenticidad del ejercicio. Cualquier actuación contraria en este sentido, aunque sea detectada en el proceso de evaluación de la prueba, que quede acreditada por parte del profesorado, dará lugar a la calificación numérica de cero, la cual no tendrá carácter de sanción, con independencia de las responsabilidades disciplinarias a que haya lugar.

Artículo 15. Originalidad de los trabajos y pruebas.

2. El plagio, entendido como la presentación de un trabajo u obra hecho por otra persona como propio o la copia de textos sin citar su procedencia y dándolos como de elaboración propia, conllevará automáticamente la calificación numérica de cero en la asignatura en la que se hubiera detectado, independientemente del resto de las calificaciones que el estudiante hubiera obtenido. Esta consecuencia debe entenderse sin perjuicio de las responsabilidades disciplinarias en las que pudieran incurrir los estudiantes que plagien.

En caso de detectar plagio o copia se aplicará el mismo criterio sancionador tanto a quien copia como a quien es copiado.

11. Entrega de la práctica

El profesor dará instrucciones adicionales sobre lo que hay que incluir en la entrega del proyecto así como los plazos para ello. En cualquier caso, el material deberá subirse al servidor web de la asignatura (void.ugr.es) y se evaluará únicamente el material subido al servidor.

Para esta práctica en concreto:

- Dentro de su carpeta **public_html** debe incluir una carpeta llamada "proyecto".
- Dentro de esa carpeta debe incluir todos los ficheros requeridos. Puede usar subdirectorios si lo estima oportuno.
- Debe existir en dicha carpeta un fichero **index.html** o **index.php** tal que, al cargarlo, se visualice la aplicación desarrollada. Por ejemplo, si el alumno tuviese como nombre de usuario **joseperez1234** el proyecto debería visualizarse al cargar la siguiente URL:

<https://void.ugr.es/~joseperez1234/proyecto>

Observaciones:

- No cumplir con alguno de los requisitos de entrega invalidará la entrega completa.
- No se admitirán entregas por ningún otro medio.
- No se admitirán entregas pasado el plazo establecido.
- La aplicación debe funcionar correctamente en el servidor **void.ugr.es**. En caso de que falle la práctica, podrá tener una calificación de "**cero**".
- Existirá un único documento pdf con toda la documentación necesaria accesible desde el pie de página del sitio web.

12. Prácticas en equipo

Esta práctica se puede desarrollar de forma individual o en pareja. La autoría debe quedar reflejada en el pie de página del sitio y en la documentación entregada así como en los ficheros fuente del proyecto. En caso de que se vaya a realizar en parejas se le debe comunicar al profesor en el plazo establecido para ello. **No comunicar este dato en tiempo y forma supondrá que la práctica deberá entregarse de forma individual obligatoriamente.**

En caso de que se haga en pareja, podrá solicitar al profesor un usuario/clave para alojarla que sea compartido por los integrantes del equipo de forma que se mantenga la privacidad en el resto de prácticas.